

Parallel Solutions for Sinkhorn Factorization

Contents

1. Introduction	1
2. State of the Art	1
3. Serial Algorithm	2
4. Parallel Algorithm	3
5. Distributed Parallel Algorithm	4
5.1. Distributed Sinkhorn-Knopp	4
6. Conclusions	5
7. References	5

1. Introduction

Sinkhorn's theorem states the following: given any square matrix A of $n \times n$ **strictly positive** elements, there are two diagonal matrices D_1, D_2 with strictly positive diagonal elements such that $D_1 A D_2$ is **double stochastic**, that is each of its rows and columns sums to one.

$$\sum_i x_{ij} = \sum_j x_{ij} = 1$$

This is also referred to as **Sinkhorn factorization**.

This theorem has many applications, most notably in optimal transport (for example computing fast approximations to Wasserstein / Earth Mover's Distance) and machine learning problems.

2. State of the Art

Researches have conducted extensive works on this topic, in particular they have studied how this algorithm could be speed up through parallelization. As said before the applications that seems to be considered more useful are in the field of machine learning, in particular in the calculation of the Word Mover's Distance. The WMD measures semantic dissimilarity between two text documents, but classically calculating it is very expensive $O(V^3 \log(V))$ where V is the number of unique words in the document. Luckily the WMD can be approximated as an Earth Mover's Distance (EMD) for which the algorithmic complexity can be reduced to $O(V^2)$ and it can be solved using the Sinkhorn-Knopp algorithm. For example [a paper by Tithi and Petrini](#) brings first a theoretical contribution, by transforming the "*dense compute-heavy kernel to a sparse compute kernel and obtain 67× speedup*". They also have shown an implementation of their algorithm using OpenMP and C++ which is 700× faster than the naive Python implementation. This program was run with 96 cores on a state of the art Intel® 4-sockets Cascade Lake machine. Another [paper by the same authors](#), a year later, provided a new version that brings additional theoretical speedups by leveraging the architecture of both conventional CPUs and emerging sparse-friendly hardware (the Intel Programmable Integrated Unified Memory Architecture aka PIUMA).

Another paper by [Sun, Luo, Jiang, Zhang and Li](#) implements an optimized version of the Sinkhorn-Knopp algorithm, named COFFEE, specifically tailored for HPC environments. They implemented various optimizations including: a redesigned column rescaling approach to improve memory locality and reduce cache misses, data blocking for cache-friendly matrix partitioning, custom SIMD vectorized micro kernels for high instruction level parallelism, and advanced MPI enhancements

such as balanced intranode Reduce and pipelined/overlapped hierarchical Ring Allreduce. They were able to achieve substantial performance improvements. Experiments on the Tianhe-1 supercomputer (up to 64 nodes/576 cores) show up to $7.5\times$ speedup on a single node and up to $2.9\times$ in multi-node settings compared to standard MPI based implementations.

3. Serial Algorithm

Stated the problem, there is a simple but effective algorithm called the Sinkhorn–Knopp algorithm (or just Sinkhorn algorithm) which takes an iterative approach, meaning that the solutions are initially approximative and subsequently improved after each iteration, and the i -th solution is derived from the previous ones.

Below is presented the pseudocode of the algorithm

```
function getError(vector)
    return max(abs(vector .- 1))
end

function sinkhorn(A, tolerance = 1e-6, maxIterations = 100)
    # Sinkhorn factorization A = D1 S D2 where D1 and D2 are diagonal and S doubly
    # stochastic using Sinkhorn-Knopp algorithm which consists in iterative rows
    # and columns sums normalizations.
    n = size(A)
    D1, D2 = vector(1,n), vector(1,n) # initialize as vectors of length n and value 1
    S = A
    iter, err = 0, +Inf
    while ((err > tolerance) && (iter < maxIterations))
        rowSums = sum(S, dims = 2) # return a vector of row sums
        D1 = D1 .* rowSums
        S = Diagonal(1 ./ rowSums) * S # Diagonal returns the diagonal matrix from a
vector
        columnSums = sum(S, dims = 1) # return a vector of column sums
        D2 = columnSums .* D2
        S = S * Diagonal(1 ./ columnSums)
        iter += 1
        err = max(getError(rowSums), getError(columnSums))
    end
    return S, Diagonal(D1), Diagonal(D2), iter
end
```

Successively we implemented this idea in C++ and measured the execution time. As expected this naive implementation is fairly slow. *Executed on a HPC Cluster with a Linux CentOS 7 Operative System and PBS Professional.*

Matrix Size (n)	Execution Time (seconds)
200	~1s
500	~15.7s
700	~43.1s
1000	~124s

4. Parallel Algorithm

Given the single process approach we can see how it is particularly suitable for parallelization as operations involving rows and columns can be done independently. As a first approach we simply parallelized the matrix multiplication using OpenMp directives.

```
DMatrix matrixMultiplyOpenmp(DMatrix A, DMatrix B)
// Multiplies two matrices A and B
{
    int n = A.size();
    DMatrix C(n, vector<double>(n, 0.0));
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < n; i++)
    {
        for (int k = 0; k < n; k++)
        {
            double temp = A[i][k];
            for (int j = 0; j < n; j++)
            {
                C[i][j] += temp * B[k][j];
            }
        }
    }
    return C;
}
```

We can also notice how the loops and the formulas were slightly rearranged in order to leverage a cache friendly access pattern, thus providing a further practical speedup. This is because in the code `C[i][j] += temp * B[k][j]` the matrix B is accessed row-wise. Since a matrix is also stored in memory row-wise, this means it is accessed sequentially and it translates to a very good cache locality.

Below we can see the execution times of this solution. We can immediately notice how it is faster than the sequential solution, but other than that we can notice the speedup offered by adding more threads. Of course, there is also a diminishing return due to the overhead brought by multi threading.

Matrix Size (n)	2 threads	4 threads	8 threads
200	~0.5s	~0.25s	~0.15s
500	~5.3s	~3.5s	~1.4s
700	~15s	~8.3s	~5s
1 000	~48s	~19.5s	~12s

During the execution we allocated as many cpus as threads with the directive `#PBS -l select=1:ncpus=8:mem=2gb`. This was done in order to make sure that threads didn't compete for each others resources and thus slow down the whole program.

We implemented another version of the algorithm based on a simple scaling operation instead of matrix multiplication. This algorithm is able to tackle very high matrix sizes in a considerable amount of time. We also noticed similar dynamics as the version above, that is benefits brought by adding more threads although with diminishing returns. The implementation is optimized using OpenMP as well and we tried to keep in mind cache locality also in this case.

Both relevant pieces of code and execution times are presented below.

```

DMatrix matrixNormalizeOpenmp(DMatrix matrix, vector<double> v, int axis)
{
    int n = matrix.size();
    DMatrix C(n, vector<double>(n, 0.0));
    if (axis == 0)
    { // Normalize along columns
        #pragma omp parallel for
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                C[i][j] = matrix[i][j] / v[j];
            }
        }
    }
    else if (axis == 1)
    { // Normalize along rows
        #pragma omp parallel for
        for (int i = 0; i < n; i++)
        {
            double v_i = v[i];
            for (int j = 0; j < n; j++)
            {
                C[i][j] = matrix[i][j] / v_i;
            }
        }
    }
    return C;
}

```

Matrix Size (n)	2 threads	4 threads	8 threads
5000	~2.5s	~1.7s	~1.3s
7500	~6.2s	~4.5s	~3.5s
10000	~10.2s	~7.3s	~7s

5. Distributed Parallel Algorithm

In a multi node context the workload needs to be distributed beforehand and there needs to be a minimal amount of communication between nodes. We decided to adopt the well known Forster's methodology also known as PCAM (Partitioning, Communication, Agglomeration, Mapping). PCAM is a simple design recipe that breaks down the process into four phases to ensure efficient parallelism, and it could generally be applied to almost any algorithm that can be parallelized.

5.1. Distributed Sinkhorn-Knopp

1. After generating a $n \times n$ matrix the algorithm proceeds by equally partitioning all the n rows among the p processes (in our case n must be divisible by p) using the function *MPI_Scatter*. The matrix is "flattened" and turned into a vector during this process for simplicity.
2. The algorithm executes the following loop for a pre-determined number of iterations

3. First each process computes independently the sums along its submatrix rows, and divides each element of the row by this sum value.
4. Then each process computes the sums along its submatrix columns. Then these vectors are summed together among all processes to obtain the global sums of the columns. This is done using *MPI_Allreduce* with the *MPI_SUM* flag. After obtaining the vector of global column sums each processes divides independently its columns by the corresponding value. After this we go back to the beginning of the loop.
5. After exiting the loop, using *MPI_Gather* all the sub matrices of each process are combined into one, as well as the *D1* and *D2* vectors that were calculated in the loop (see the introduction section to look up what these variable represent)

In the table below are noted the execution times of the program described above. First we can notice how it is a bit slower compared to the purely openmp version. This is because the MPI processes are scattered among different nodes so naturally communication creates an overhead, but it must be said that a multi node architecture allows to tackle inputs that may not fit in the memory or be able to be processed by a single node. We can also notice how by adding more processes the computation becomes faster, for example the execution times with 4 processes are half compared to those with only 2. However with 8 processes there is no longer a speedup. We suspect that the slowdown caused by communication becomes more relevant than the theoretical speedup brought by adding more processes.

Note: the execution was done using `-l place=scatter` and 4 openmp threads

Matrix Size (<i>n</i>)	2 MPI processes	4 MPI processes	8 MPI processes
5000	~3.3s	~2.2s	~2.4s
10000	~15s	~8.8s	~9.7s
12000	~28s	~14s	~15s

6. Conclusions

During this project we have learnt how to implement a parallel algorithm in cpp using both openmp directives and MPI functions by implementing working solutions for the Sinkhorn-Knopp algorithm.

In particular, we first explored how to transform a computationally intensive iterative matrix scaling method into efficient parallel version using OpenMP. We implemented a shared memory approach that leverages compiler directives to parallelize loops and distribute workload among threads while carefully managing synchronization and avoiding race conditions, all while keeping an eye on cache efficiency. In contrast, with MPI, we developed a distributed solution in which data is partitioned across multiple processes, requiring explicit communication through message passing to ensure correctness at each iteration.

Throughout this process, we gained practical experience in workload decomposition, process communication and performance optimization. Overall, this project strengthened our knowledge of parallel computing concepts and provided hands on experience in applying them to a real numerical algorithm.

7. References

1. [Sinkhorn Theorem](#)

2. Earth Mover Distance
3. Tithi, Petrini n.1
4. Tithi, Petrini n.2
5. Sun, Luo, Jiang, Zhang and Li